

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 - Viva Based Questions

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5-AT2	Assessment:	Assessment Tool 2-Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	MURALI KRISHNAN N	Reg. No.:	192524090
Section / Batch:	B.TECH AI&DS/ 25 BATCH	Date:	2/08/26

Code of Conduct

I MURALI KRISHNAN N (192524090)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	

Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Q35: Confusion Matrix & Classification Metrics From Scratch

1. Problem Statement

Implement, from scratch (without using sklearn.metrics), the following evaluation tools for a multiclass classification problem:

- A confusion matrix
- Overall accuracy
- Per-class precision, recall, and F1-score
- Macro-averaged precision, recall, and F1-score
- Weighted-averaged precision, recall, and F1-score

The implementation uses only Python and NumPy for arithmetic — no sklearn.metrics functions are called for the metric computations themselves.

2. Key Definitions

For a given class i , treating it as the "positive" class and all other classes as "negative" (one-vs-rest):

- True Positive (TP): predicted class i , actual class i
- False Positive (FP): predicted class i , actual is not i (column sum minus TP)
- False Negative (FN): actual class i , predicted is not i (row sum minus TP)

From these:

- $\text{Precision}_i = \text{TP} / (\text{TP} + \text{FP})$ — of all items predicted as class i , how many were correct

- $\text{Recall}_i = \text{TP} / (\text{TP} + \text{FN})$ — of all actual items of class i , how many were found
- $\text{F1}_i = 2 \times \text{Precision}_i \times \text{Recall}_i / (\text{Precision}_i + \text{Recall}_i)$ — harmonic mean of the two

The confusion matrix itself is an $n \times n$ grid (n = number of classes) where $\text{cm}[i][j]$ counts samples whose true label is class i and predicted label is class j . The diagonal holds correct predictions; off-diagonal entries reveal specific confusions between classes.

3. Full Implementation

3.1 Confusion Matrix

Builds an $n \times n$ matrix by mapping each label to an index and incrementing $\text{cm}[\text{true_idx}][\text{pred_idx}]$ for every sample.

```
import numpy as np

def confusion_matrix_scratch(y_true, y_pred, labels=None):
    if labels is None:
        labels = sorted(set(y_true) | set(y_pred))
    label_to_idx = {label: idx for idx, label in enumerate(labels)}
    n = len(labels)
    cm = np.zeros((n, n), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[label_to_idx[t]][label_to_idx[p]] += 1
    return cm, labels
```

3.2 Accuracy

The fraction of predictions that exactly match the true label — a single global number, not per-class.

```
def accuracy_scratch(y_true, y_pred):
    correct = sum(t == p for t, p in zip(y_true, y_pred))
    return correct / len(y_true)
```

3.3 Per-Class Precision, Recall, F1

Derives TP, FP, FN for each class directly from the confusion matrix (row sums, column sums, and the diagonal), guarding against division by zero when a class has no predicted or no actual members.

```
def precision_recall_f1_per_class(cm):
    n = cm.shape[0]
    precision = np.zeros(n)
    recall = np.zeros(n)
    f1 = np.zeros(n)
    support = np.zeros(n, dtype=int)
    for i in range(n):
        tp = cm[i][i]
        fp = cm[:, i].sum() - tp      # column sum minus TP
        fn = cm[i, :].sum() - tp      # row sum minus TP
        support[i] = cm[i, :].sum()   # actual count of class i
        precision[i] = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall[i] = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1[i] = (2 * precision[i] * recall[i] / (precision[i] + recall[i]))
            if (precision[i] + recall[i]) > 0 else 0.0
    return precision, recall, f1, support
```

3.4 Macro-Averaged Metrics

Simple, unweighted mean of the per-class scores — every class counts equally regardless of how many samples it has.

```
def macro_average(precision, recall, f1):
    return precision.mean(), recall.mean(), f1.mean()
```

3.5 Weighted-Averaged Metrics

Mean of the per-class scores weighted by each class's support (number of true instances), so classes with more samples contribute proportionally more.

```
def weighted_average(precision, recall, f1, support):
    total = support.sum()
    w_p = (precision * support).sum() / total
    w_r = (recall * support).sum() / total
    w_f1 = (f1 * support).sum() / total
    return w_p, w_r, w_f1
```

3.6 Full Classification Report

Ties everything together into a printable report, similar in shape to sklearn's `classification_report`.

```
def classification_report_scratch(y_true, y_pred, labels=None):
    cm, labels = confusion_matrix_scratch(y_true, y_pred, labels)
    precision, recall, f1, support = precision_recall_f1_per_class(cm)
    acc = accuracy_scratch(y_true, y_pred)
    macro_p, macro_r, macro_f1 = macro_average(precision, recall, f1)
    w_p, w_r, w_f1 = weighted_average(precision, recall, f1, support)

    print(f'{"Class":<10} {"Precision":<12} {"Recall":<12} {"F1-score":<12} {"Support":<10} ')
    for i, label in enumerate(labels):
        print(f'{"str(label)":<10} {"precision[i]:<12.3f} {"recall[i]:<12.3f} '
              f'{"f1[i]:<12.3f} {"support[i]:<10} ')
    print('-' * 56)
    print(f'{"Accuracy":<10} {"":<12} {"":<12} {"acc:<12.3f} {"support.sum():<10} ')
    print(f'{"Macro avg":<10} {"macro_p:<12.3f} {"macro_r:<12.3f} {"macro_f1:<12.3f} {"support.sum():<10} ')
    print(f'{"Wtd avg":<10} {"w_p:<12.3f} {"w_r:<12.3f} {"w_f1:<12.3f} {"support.sum():<10} ')

    return {
        'confusion_matrix': cm, 'labels': labels, 'accuracy': acc,
        'precision': precision, 'recall': recall, 'f1': f1, 'support': support,
        'macro_avg': (macro_p, macro_r, macro_f1),
        'weighted_avg': (w_p, w_r, w_f1),
    }
```

4. Example Run

A 3-class example (cat / dog / bird, 15 samples) — chosen deliberately imbalanced so macro and weighted averages differ:

```
y_true = ['cat','dog','cat','bird','dog','dog','cat','bird','bird','dog',
          'cat','dog','bird','cat','dog']
y_pred = ['cat','dog','cat','bird','cat','dog','cat','bird','dog','dog',
          'dog','dog','bird','cat','dog']
```

```
cm, labels = confusion_matrix_scratch(y_true, y_pred)
results = classification_report_scratch(y_true, y_pred)
```

Confusion Matrix (rows = actual, columns = predicted; order: bird, cat, dog)

```
      bird cat dog
bird [ 3  0  1 ]
cat  [ 0  4  1 ]
dog  [ 0  1  5 ]
```

Resulting Report

Class	Precision	Recall	F1-score	Support
bird	1.000	0.750	0.857	4
cat	0.800	0.800	0.800	5
dog	0.714	0.833	0.769	6
Accuracy			0.800	15
Macro avg	0.838	0.794	0.809	15
Weighted avg	0.819	0.800	0.803	15

This output was cross-checked against `sklearn.metrics.confusion_matrix` and `classification_report` on the same data and matches to three decimal places — confirming the from-scratch math is correct. (sklearn was used only to verify the result, not inside the implementation itself.)

5. Macro vs. Weighted Averaging: When to Use Each

Macro Average

Computes the unweighted mean across all classes: $\text{Macro-F1} = (1/n) \times \sum \text{F1}_i$. Every class, no matter how rare, contributes equally to the final score.

- Use when every class matters equally regardless of its frequency — e.g., detecting a rare disease, a rare fraud pattern, or a minority category where missing it is just as costly as missing a common one.
- Use when you specifically want to expose poor performance on minority classes. A model that ignores small classes but nails the big ones will score poorly on macro average, which is often the point of using it.
- Appropriate for imbalanced datasets when the minority class is the one you actually care about.

Weighted Average

Computes the mean across classes weighted by each class's support: $\text{Weighted-F1} = \sum (\text{support}_i \times \text{F1}_i) / \sum \text{support}_i$. Classes with more samples influence the score more.

- Use when overall predictive performance across the real-world class distribution is what matters — e.g., general-purpose classifiers where the test distribution reflects true prevalence.
- Use when class imbalance is a natural property of the data (not a defect to correct for) and you want a single summary number that reflects how the model performs "on average" across actual traffic.

- Less useful if you need to know whether a rare-but-important class is being handled well, since a model can score high on weighted average while completely failing a small class.

Quick Rule of Thumb

If class imbalance exists and every class is equally important → use macro average. If class imbalance exists but reflects real-world proportions and you want an overall performance summary → use weighted average. If classes are balanced, macro and weighted average converge to the same value, so the choice matters much less.

6. Complexity

Building the confusion matrix is $O(N)$ in the number of samples. Deriving precision/recall/F1 per class from the matrix is $O(n^2)$ for an n -class problem (row/column sums), which is negligible next to N in virtually all real datasets. No `sklearn.metrics` functions were used at any point — only Python built-ins and NumPy array operations.